A well-specified linear regression model

Tomas Kysela

Introduction

A regression model may suffer from various issues. When constructing such a model, it is important to check for assumptions such as linearity, independence of errors, normality of residuals, homoscedasticity and the absence of multicollinearity. Additionally, there are other potential issues, such as irrelevant variables, which can reduce model performance. In this assignment you will aim to achieve the best possible linear model using the techniques you have already learned.

Input data

In this assignment, you will work with a dataset that contains 200 samples, 10 features $x_1, \ldots, x_{10}$ and one dependent variable y. Your general task is to develop the best possible linear model. One that is both simple and interpretable while still performing well. You are expected to correctly interpret the model. Assume that outliers are not expected (if they are present they should be removed), and our primary focus is on identifying common trends rather than a few exceptional samples.

Load the necessary libraries and the dataset

```r
library(car) # use vif function for multicollinearity detection
```

```
## Loading required package: carData
```

```r
library(lmtest) # use bptest for homoscedasticity test
```

```
## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
library(glmnet) # shrinked linear models
```

```
## Loading required package: Matrix

## Loaded glmnet 4.1-8
```

```r
library(MASS)

d <- read.csv("lreg_data.csv")
```

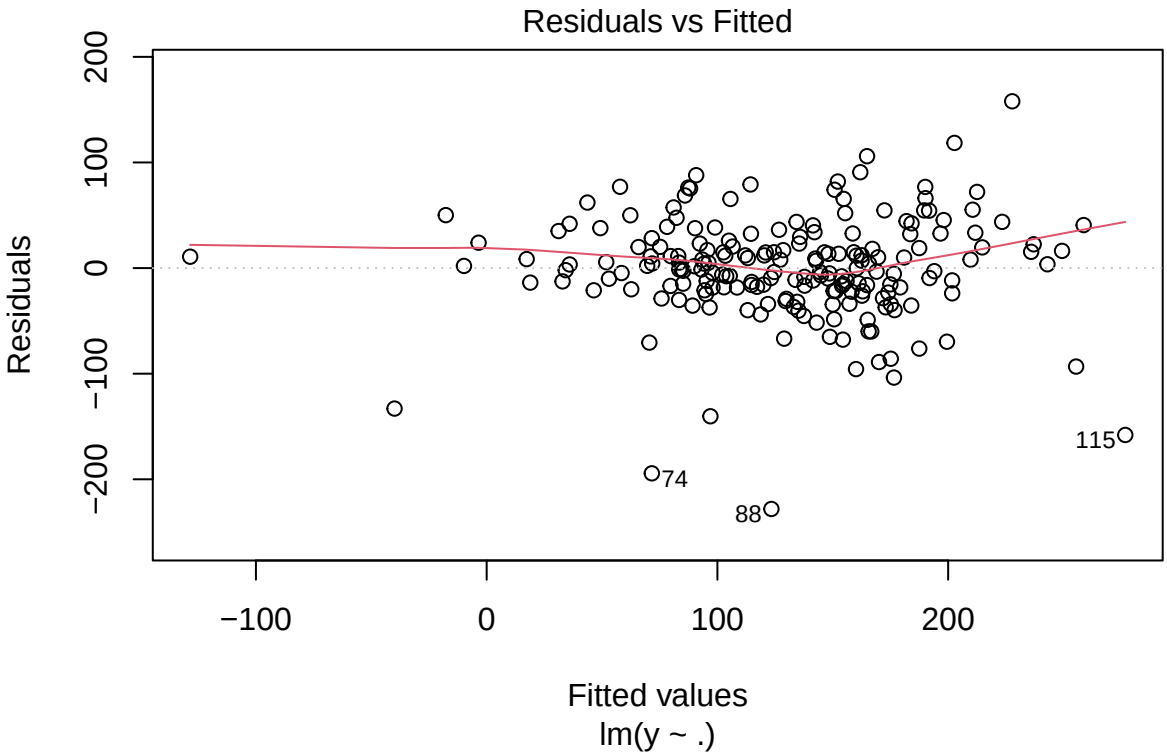
The full linear model

Let us start with a full linear model. The first task is to evaluate the model based on its first summary. Then, check its assumptions and detect its potential issues. Your task is to evaluate the plots and tests performed below, you should also further develop the tests (propose their alternatives).

```r
#### improve the code below so that you can verbally explain the full model ####

# construct the model
full_lm <- lm(y ~ ., d)
summary(full_lm)
```

```
##
## Call:
## lm(formula = y ~ ., data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -228.093  -21.041    1.894   24.501  157.924
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 148.6082    51.0869   2.909 0.004061 **
## x1           -0.3404     0.3662  -0.929 0.353880
## x2            0.5223     0.2599   2.010 0.045906 *
## x3           -0.4873     0.6532  -0.746 0.456665
## x4           -2.5976     0.7574  -3.429 0.000742 ***
## x5            1.1959     0.5055   2.366 0.019004 *
## x6           -0.3778     1.8612  -0.203 0.839357
## x7           -0.0174     0.2389  -0.073 0.942005
## x8           -0.6565     0.4402  -1.491 0.137543
## x9            1.3295     0.5107   2.603 0.009973 **
## x10           1.0228     0.2342   4.367 2.08e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 50.29 on 189 degrees of freedom
## Multiple R-squared:  0.5803, Adjusted R-squared:  0.5581
## F-statistic: 26.13 on 10 and 189 DF,  p-value: < 2.2e-16
```

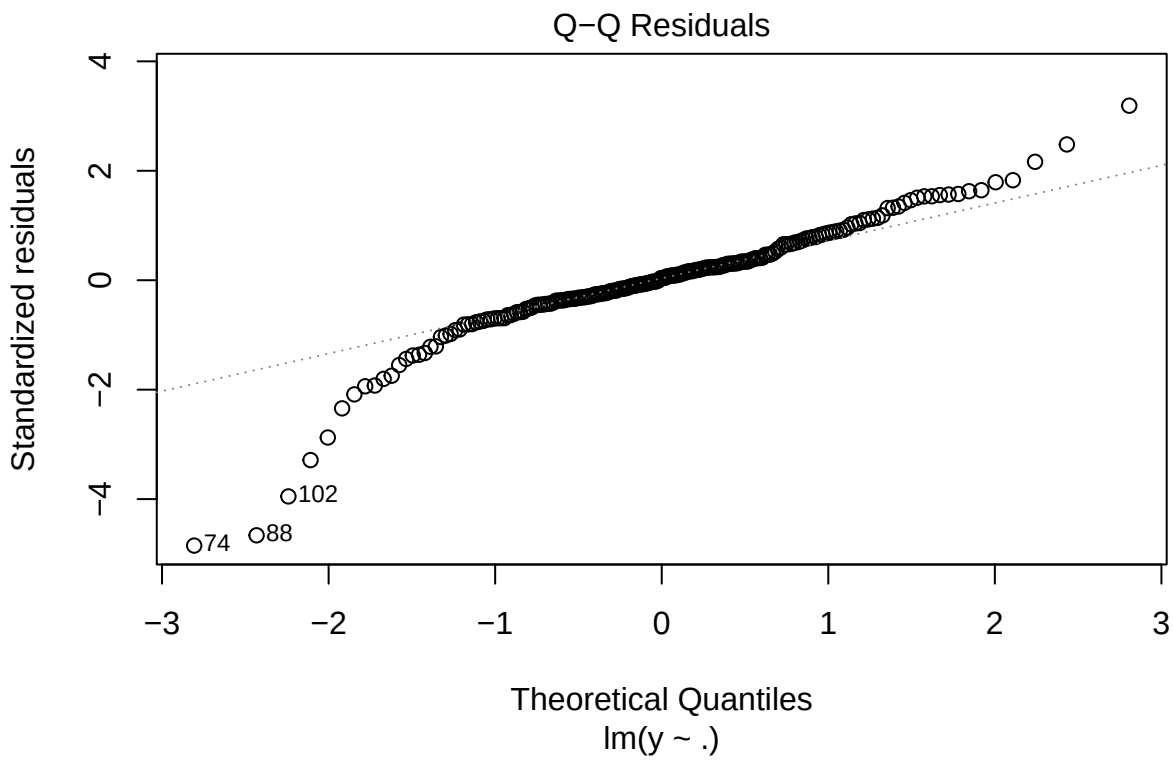
```r
# check for the issues

# start with the residual plot
plot(full_lm, which=1)
```




```r
# perform a homoscedasticity test
bptest(full_lm)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  full_lm
## BP = 34.971, df = 10, p-value = 0.0001263
```

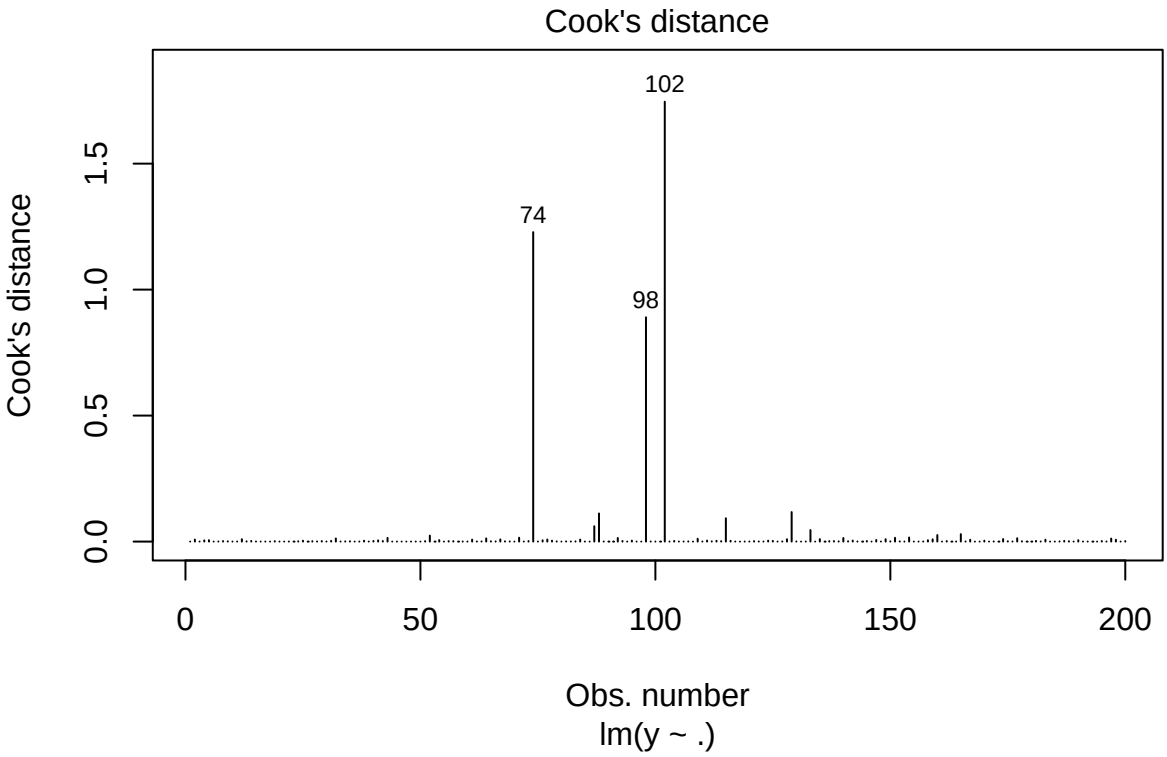
```r
# normality of residuals? Q-Q plot and normality test
plot(full_lm, which = 2)
```




```r
shapiro.test(residuals(full_lm))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(full_lm)
## W = 0.93745, p-value = 1.384e-07
```

```r
# search for outliers
plot(full_lm, which = 4)
```




```r
which(cooks.distance(full_lm) > 4 / (nrow(d) - length(coef(full_lm))))
```

```
##  52  74  87  88  98 102 115 129 133 160 165
##  52  74  87  88  98 102 115 129 133 160 165
```

```r
# detect potential multicollinearity
vif(full_lm)
```

```
##        x1        x2        x3        x4        x5        x6        x7        x8
##  1.057418 10.268698  2.419465  2.143518 10.211986  1.032185  1.036711  1.541193
##        x9       x10
##  1.046448  1.050156
```

Add your verbal summary here (1p): Residual plot shows us, that model is moved towards points like 74 and 88, thus the trend of residual distance is on the whole dataset above zero.

Q-Q plot tells us, that from -1 std deviation to 2 std deviations residuals follow normal distribution. Points deviating from line on both ends means, the distribution of residuals is heavy-tailed on both sides

Cook's distance shows us, that points 74, 98 and 102 have great impact on our model and skews the model towards them. This partly matches residual plot.

Finally variables x2 and x5 have high VIF, meaning these variables should be removed from our model.

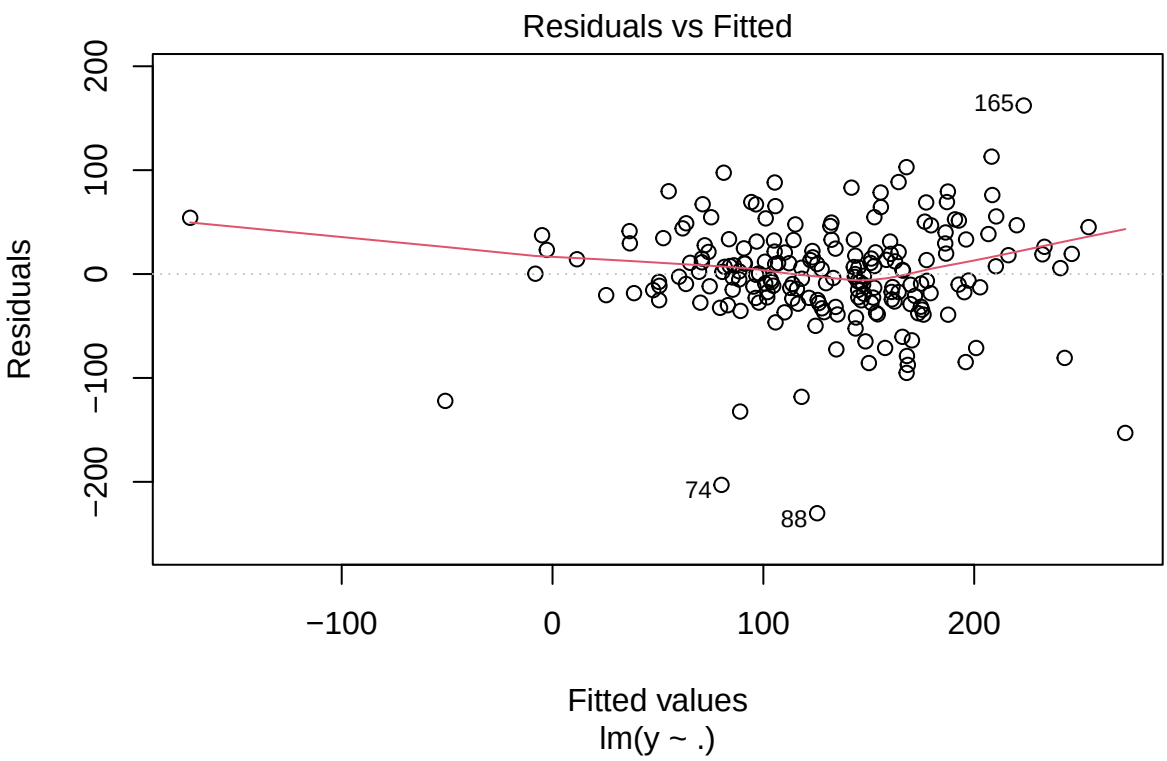
Remove the issues from the full model

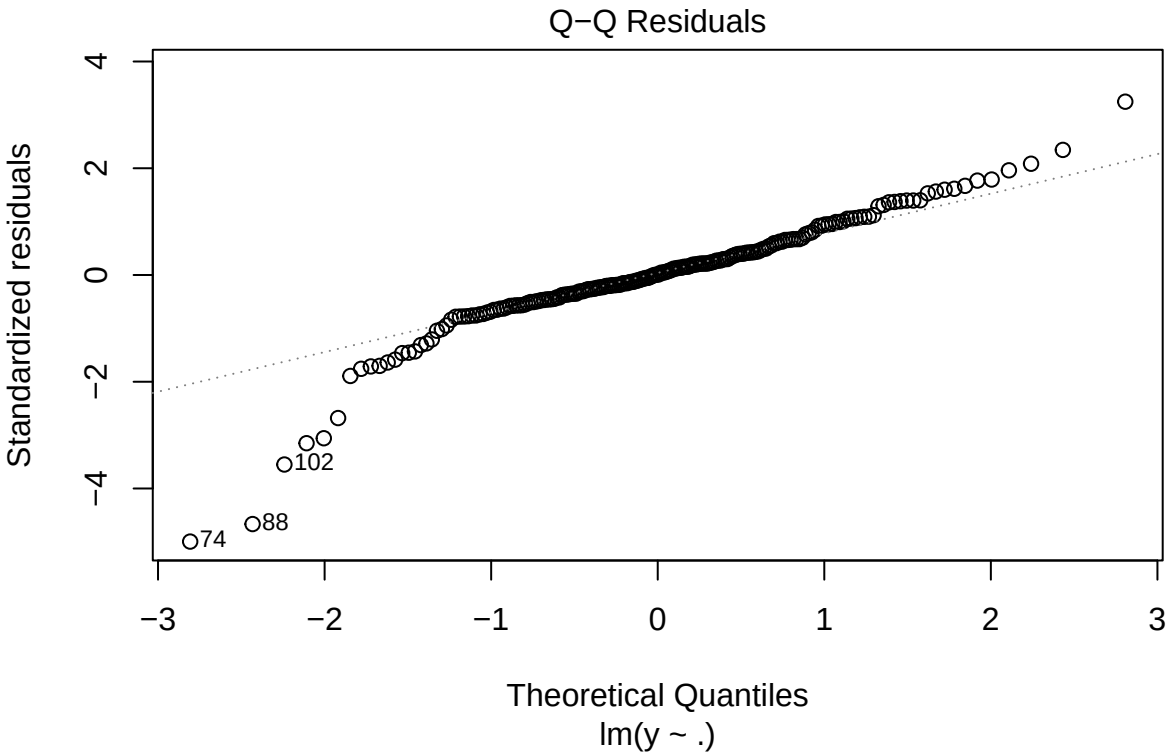
Now deal with the detected issues in the right order. Build an alternative model and show that it works better than the full model. Finally, verbally justify all the decisions that you made and explain their effects.

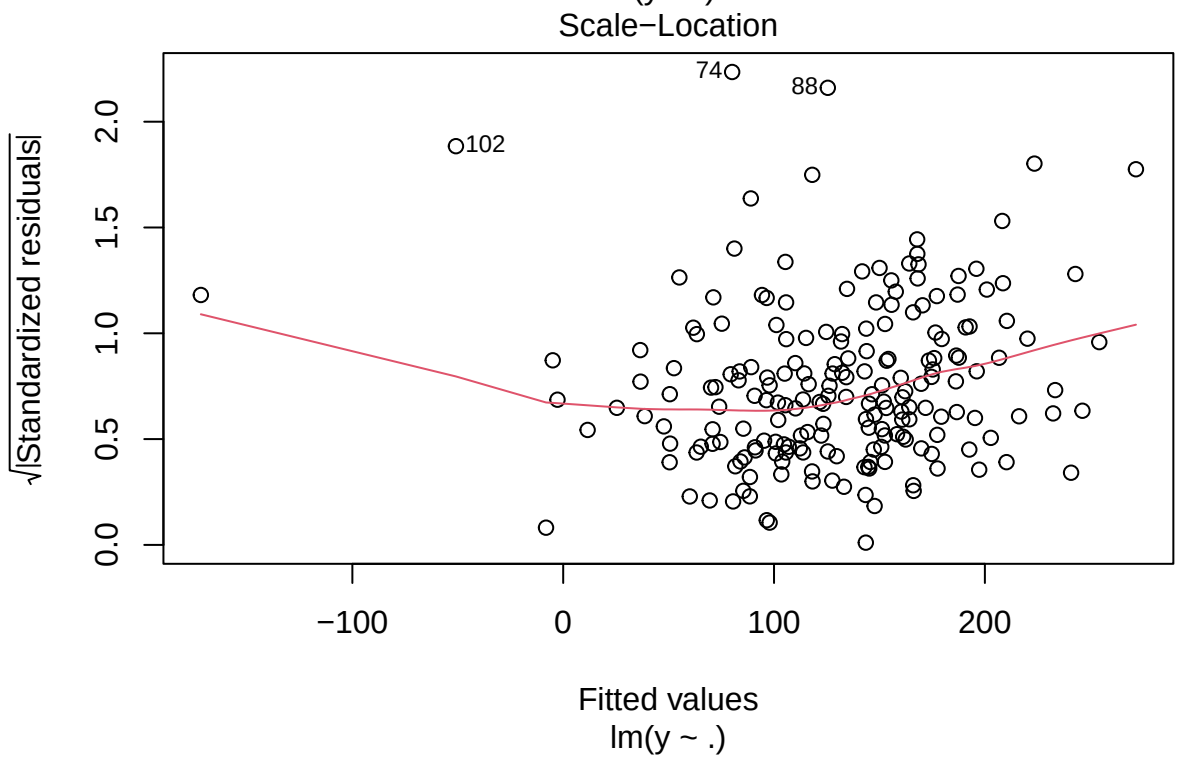
```r
#### add your code here (1p) ####
ds <- subset(d, select = -c(x2))
mod_lms <- lm(y ~ ., ds)
summary(mod_lms)
```

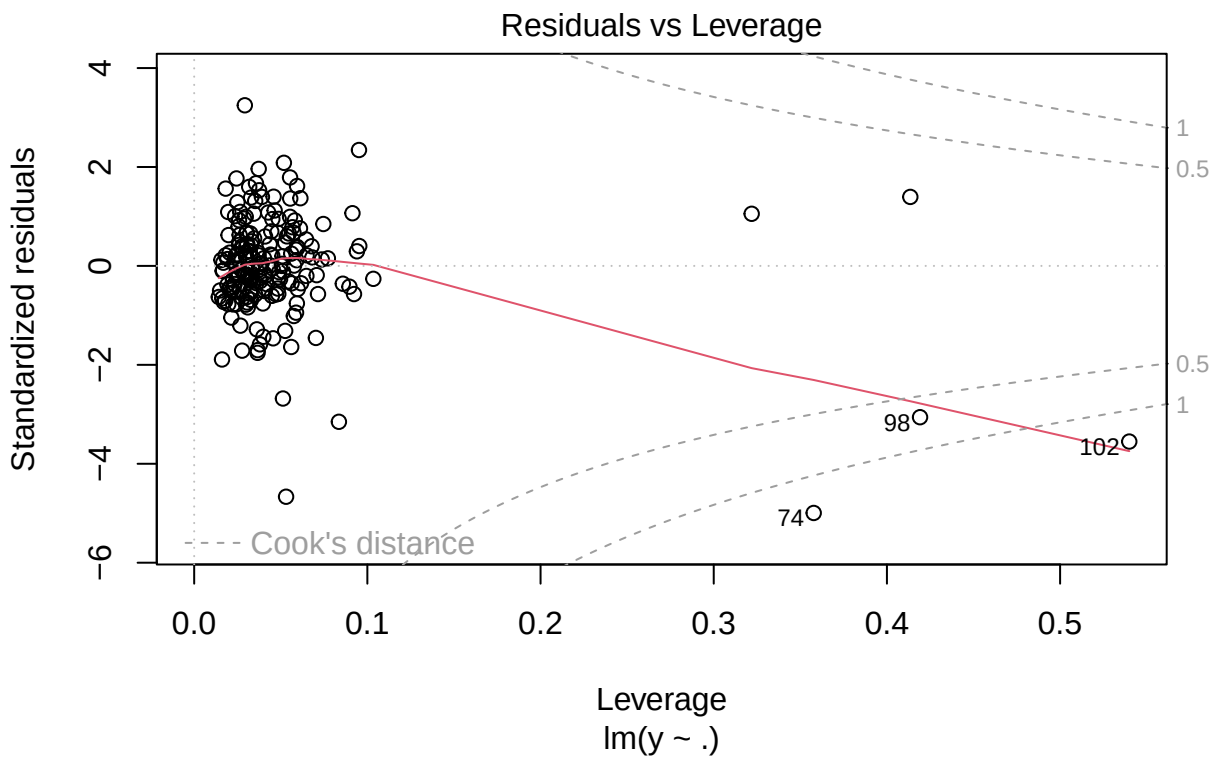
```
##
## Call:
## lm(formula = y ~ ., data = ds)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -230.196  -22.915    0.438   26.600  162.186
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 143.17538   51.42158   2.784 0.005906 **
## x1           -0.32575    0.36908  -0.883 0.378571
## x3           -0.71481    0.64849  -1.102 0.271735
## x4           -2.85650    0.75234  -3.797 0.000197 ***
## x5            2.15908    0.16183  13.342  < 2e-16 ***
## x6           -0.09567    1.87068  -0.051 0.959266
## x7           -0.07275    0.23915  -0.304 0.761312
## x8           -0.51798    0.43825  -1.182 0.238710
## x9            1.38413    0.51407   2.693 0.007725 **
## x10           0.97376    0.23481   4.147 5.07e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 50.69 on 190 degrees of freedom
## Multiple R-squared:  0.5713, Adjusted R-squared:  0.551
## F-statistic: 28.14 on 9 and 190 DF,  p-value: < 2.2e-16
```

```r
plot(mod_lms)
```




Q−Q Residuals
Standardized residuals
Theoretical Quantiles
lm(y ~ .)
102
88
74

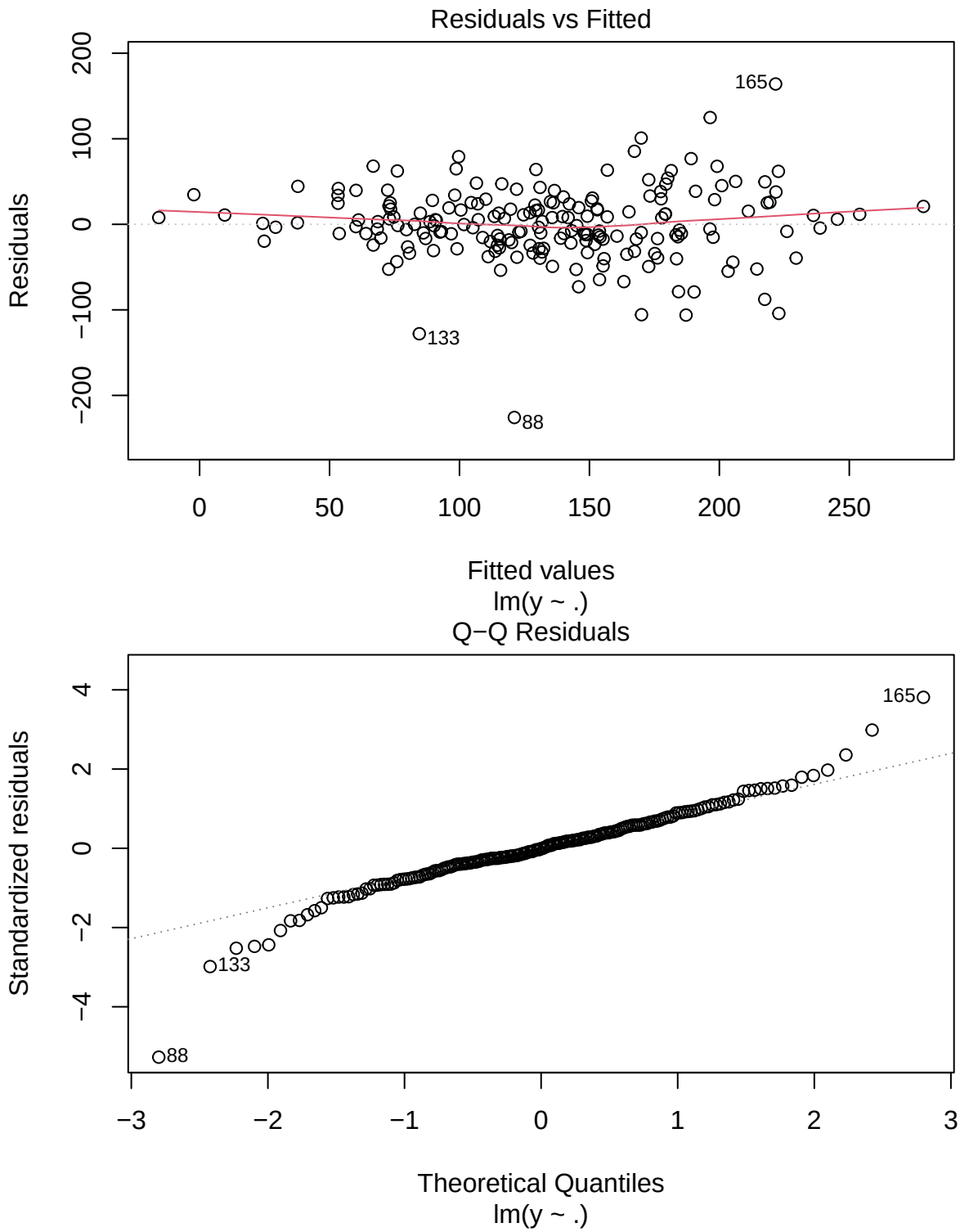

Scale−Location
√|Standardized residuals|
Fitted values
lm(y ~ .)
74
88
102

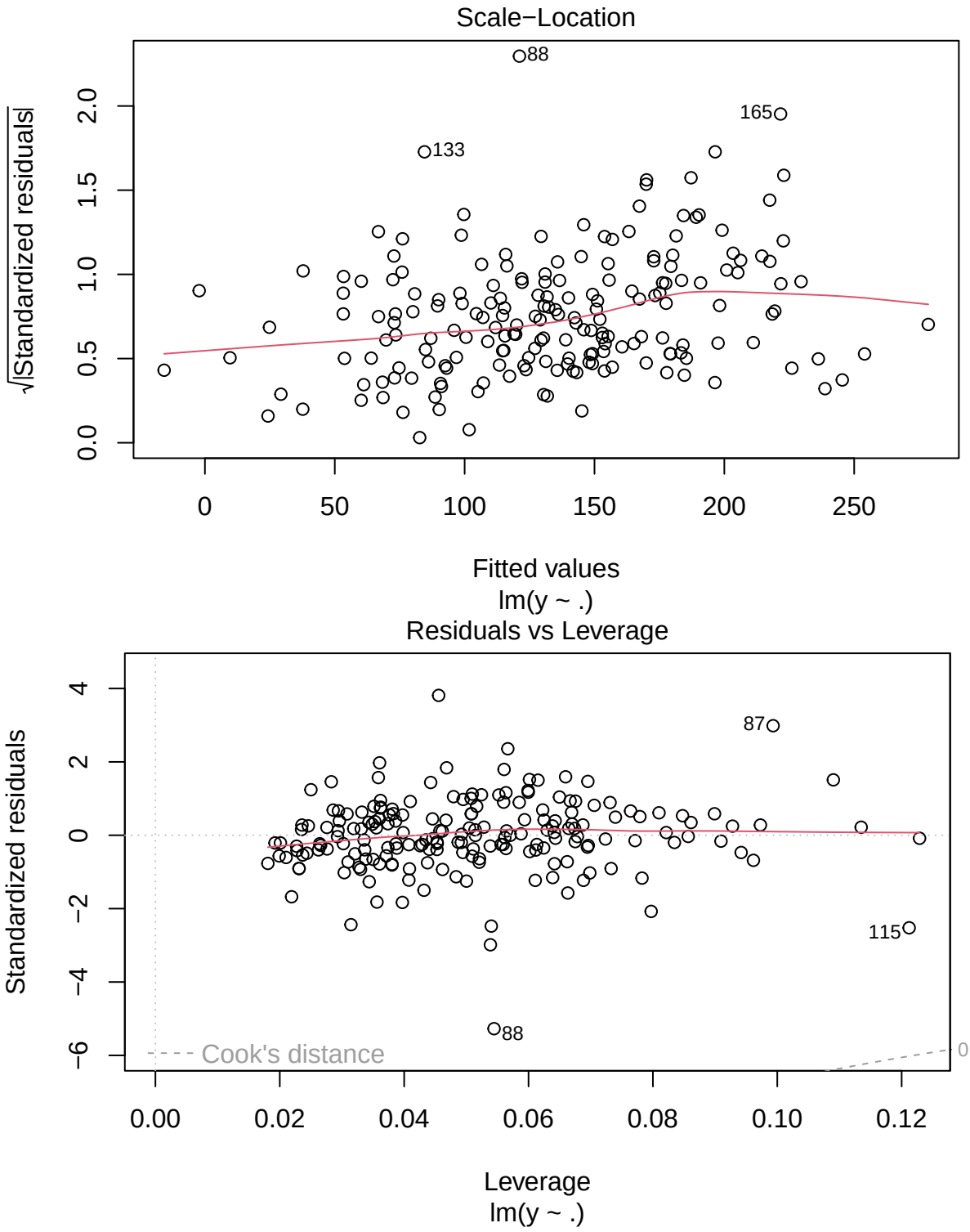



```r
d1 <- ds[-c(71, 74, 102, 98, 129), ]
mod_lm1 <- lm(y ~ ., d1)
summary(mod_lm1)
```

```
##
## Call:
## lm(formula = y ~ ., data = d1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -225.800  -19.912   -0.259   24.842  164.036
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 80.68135   60.54903   1.332  0.18434
## x1          -0.39266    0.32107  -1.223  0.22290
## x3           3.46672    1.15180   3.010  0.00298 **
## x4           0.56753    1.03833   0.547  0.58533
## x5           2.22400    0.14993  14.834  < 2e-16 ***
## x6           0.25878    1.64420   0.157  0.87511
## x7           0.12481    0.21246   0.587  0.55762
## x8          -1.81558    0.63471  -2.860  0.00472 **
## x9           1.23932    0.45099   2.748  0.00659 **
## x10          0.06509    0.28541   0.228  0.81987
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 44.05 on 185 degrees of freedom
## Multiple R-squared:  0.6016, Adjusted R-squared:  0.5822
## F-statistic: 31.04 on 9 and 185 DF,  p-value: < 2.2e-16
```

```r
plot(mod_lm1)
```



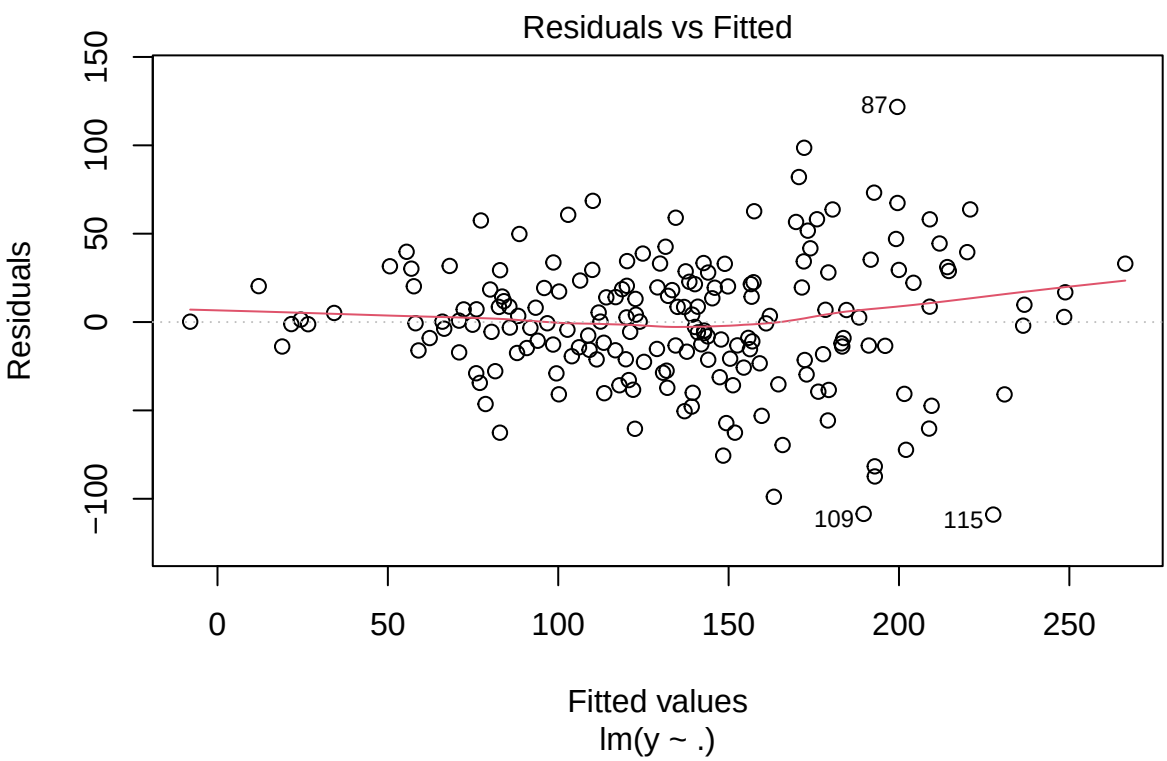


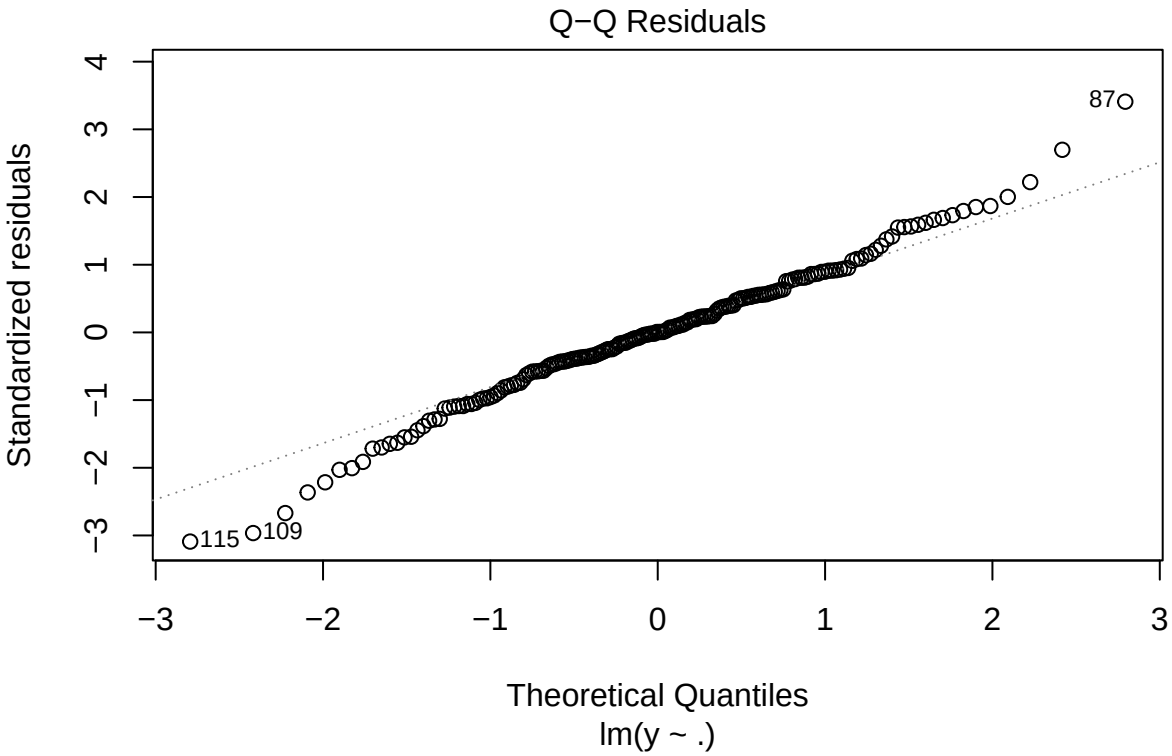

```r
d2 <- ds[-c(88, 133, 165, 71, 74, 102, 98, 129), ]
mod_lm2 <- lm(y ~ ., d2)
summary(mod_lm2)
```

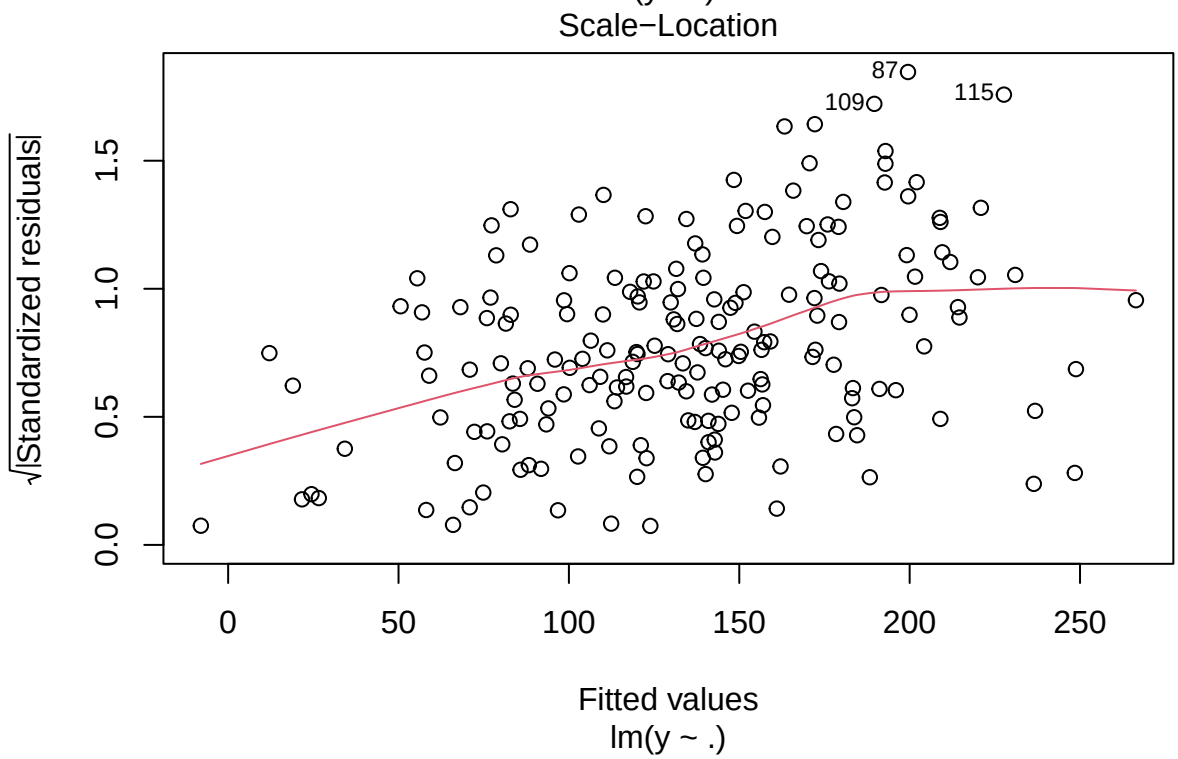
```
##
## Call:
## lm(formula = y ~ ., data = d2)
```

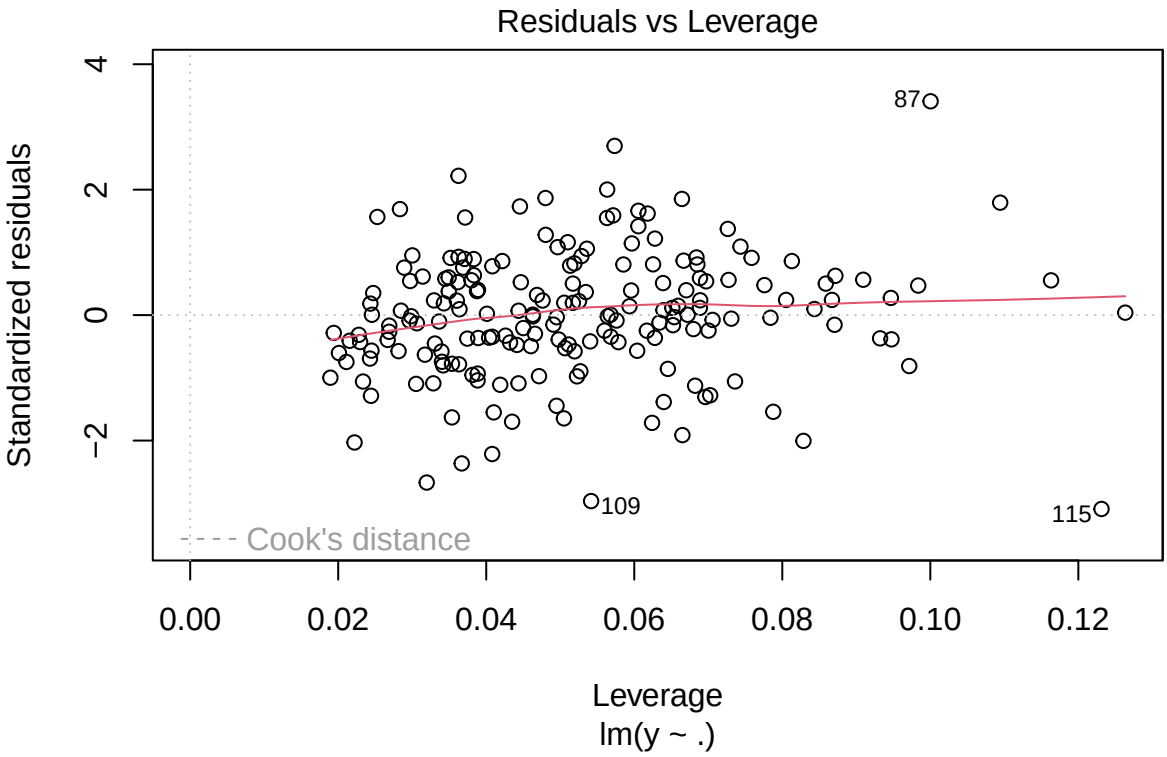
```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -108.945  -19.699    0.204   21.446  121.739
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 101.61671   52.72492   1.927 0.055499 .
## x1           -0.41185    0.27508  -1.497 0.136072
## x3            3.64744    0.98628   3.698 0.000287 ***
## x4            0.17585    0.89296   0.197 0.844105
## x5            2.04140    0.13017  15.682  < 2e-16 ***
## x6           -0.83121    1.43280  -0.580 0.562542
## x7            0.04681    0.18227   0.257 0.797617
## x8           -1.57163    0.54717  -2.872 0.004560 **
## x9            0.96601    0.38708   2.496 0.013463 *
## x10           0.06983    0.24454   0.286 0.775542
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 37.64 on 182 degrees of freedom
## Multiple R-squared:  0.6557, Adjusted R-squared:  0.6386
## F-statistic: 38.51 on 9 and 182 DF,  p-value: < 2.2e-16
```

```r
plot(mod_lm2)
```




Q−Q Residuals
Standardized residuals
Theoretical Quantiles
lm(y ~ .)
87
115
109


Scale−Location
√|Standardized residuals|
Fitted values
lm(y ~ .)
87
109
115




```r
vif(mod_lm2)
```

```
##       x1       x3       x4       x5       x6       x7       x8       x9
## 1.049996 5.073595 5.095545 1.031227 1.033691 1.055182 1.060156 1.057260
##      x10
## 1.012739
```

Add your verbal summary here (1p): First I remove variables with VIF > 10. Next I removed data-points with biggest leverage (71, 74, 102, 98, 129). After that I remove biggest outliers such as 88, 133, 165.

Feature selection

The goal now is to perform feature selection and remove irrelevant variables. Compare several different feature selection methods and choose the most appropriate one. Demonstrate that applying feature selection to the full model yields very different results compared to applying it to an alternative model without issues. Finally, explain the selected model.

```r
#### add your code here (1p) ####
print("full manual")
```

```
## [1] "full manual"
```

```r
lm.full.exclude <- lm(y ~ . -x1 -x3 -x6 -x7 -x8, data = d)
print("y ~ x2 + x4 + x5 + x9 + x10")
```

```
## [1] "y ~ x2 + x4 + x5 + x9 + x10"
```

```r
print("alt manual")
```

```
## [1] "alt manual"
```

```r
lm.alt.exclude <- lm(y ~ . -x1 -x4 -x6 -x7 -x10, data = d2)
print("y ~ x2 + x3 + x5 + x8 + x9")
```

```
## [1] "y ~ x2 + x3 + x5 + x8 + x9"
```

```r
print("full step forward")
```

```
## [1] "full step forward"
```

```r
lm.full.stepfor <- stepAIC(full_lm, direction="forward", trace=0)
lm.full.stepfor$anova
```

```
## Stepwise Model Path
## Analysis of Deviance Table
##
## Initial Model:
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
## Final Model:
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
##
##   Step Df Deviance Resid. Df Resid. Dev      AIC
## 1                        189   478049.9 1577.831
```

```r
print("alt step forward")
```

```
## [1] "alt step forward"
```

```r
lm.alt.stepfor <- stepAIC(mod_lm2, direction="forward", trace=0)
lm.alt.stepfor$anova
```

```
## Stepwise Model Path
## Analysis of Deviance Table
##
## Initial Model:
## y ~ x1 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
## Final Model:
## y ~ x1 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
##
##   Step Df Deviance Resid. Df Resid. Dev     AIC
## 1                        182     257909 1402.95
```

```r
print("full step backward")
```

```
## [1] "full step backward"
```

```r
lm.full.stepback <- stepAIC(full_lm, direction="backward", trace=0)
lm.full.stepback$anova
```

```
## Stepwise Model Path
## Analysis of Deviance Table
##
## Initial Model:
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
```

```
## Final Model:
## y ~ x2 + x4 + x5 + x8 + x9 + x10
##
##
##   Step Df   Deviance Resid. Df Resid. Dev      AIC
## 1                          189   478049.9 1577.831
## 2 - x7  1   13.42227       190   478063.4 1575.836
## 3 - x6  1  111.98462       191   478175.3 1573.883
## 4 - x3  1 1458.69634       192   479634.0 1572.492
## 5 - x1  1 1923.42017       193   481557.5 1571.293
```

```r
print("alt step backward")
```

```
## [1] "alt step backward"
```

```r
lm.alt.stepback <- stepAIC(mod_lm2, direction="backward", trace=0)
lm.alt.stepback$anova
```

```
## Stepwise Model Path
## Analysis of Deviance Table
##
## Initial Model:
## y ~ x1 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
## Final Model:
## y ~ x1 + x3 + x5 + x8 + x9
##
##
##    Step Df  Deviance Resid. Df Resid. Dev      AIC
## 1                          182   257909.0 1402.950
## 2  - x4  1  54.95432       183   257963.9 1400.991
## 3  - x7  1  75.22841       184   258039.2 1399.047
## 4 - x10  1 103.74222       185   258142.9 1397.124
## 5  - x6  1 455.42019       186   258598.3 1395.463
```

```r
print("full step both")
```

```
## [1] "full step both"
```

```r
lm.full.stepboth <- stepAIC(full_lm, direction="both", trace=0)
lm.full.stepboth$anova
```

```
## Stepwise Model Path
## Analysis of Deviance Table
##
## Initial Model:
## y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
## Final Model:
## y ~ x2 + x4 + x5 + x8 + x9 + x10
##
##
##   Step Df   Deviance Resid. Df Resid. Dev      AIC
## 1                          189   478049.9 1577.831
## 2 - x7  1   13.42227       190   478063.4 1575.836
## 3 - x6  1  111.98462       191   478175.3 1573.883
## 4 - x3  1 1458.69634       192   479634.0 1572.492
```

```
## 5 - x1  1 1923.42017       193   481557.5 1571.293
```

```r
print("alt step both")
```

```
## [1] "alt step both"
```

```r
lm.alt.stepboth <- stepAIC(mod_lm2, direction="both", trace=0)
lm.alt.stepboth$anova
```

```
## Stepwise Model Path
## Analysis of Deviance Table
##
## Initial Model:
## y ~ x1 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
##
## Final Model:
## y ~ x1 + x3 + x5 + x8 + x9
##
##
##    Step Df  Deviance Resid. Df Resid. Dev      AIC
## 1                          182   257909.0 1402.950
## 2  - x4  1  54.95432       183   257963.9 1400.991
## 3  - x7  1  75.22841       184   258039.2 1399.047
## 4 - x10  1 103.74222       185   258142.9 1397.124
## 5  - x6  1 455.42019       186   258598.3 1395.463
```

```r
anova(lm.full.exclude, lm.full.stepfor, lm.full.stepback, lm.full.stepboth)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ (x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10) - x1 -
##     x3 - x6 - x7 - x8
## Model 2: y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
## Model 3: y ~ x2 + x4 + x5 + x8 + x9 + x10
## Model 4: y ~ x2 + x4 + x5 + x8 + x9 + x10
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1    194 494847
## 2    189 478050  5   16797.2 1.3282 0.2539
## 3    193 481557 -4   -3507.5 0.3467 0.8461
## 4    193 481557  0       0.0
```

```r
anova(lm.alt.exclude, lm.alt.stepfor, lm.alt.stepback, lm.alt.stepboth)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ (x1 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10) - x1 - x4 -
##     x6 - x7 - x10
## Model 2: y ~ x1 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
## Model 3: y ~ x1 + x3 + x5 + x8 + x9
## Model 4: y ~ x1 + x3 + x5 + x8 + x9
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1    187 261731
## 2    182 257909  5    3821.8 0.5394 0.7462
## 3    186 258598 -4    -689.3 0.1216 0.9746
## 4    186 258598  0       0.0
```

```r
summary(lm.full.stepback)
```

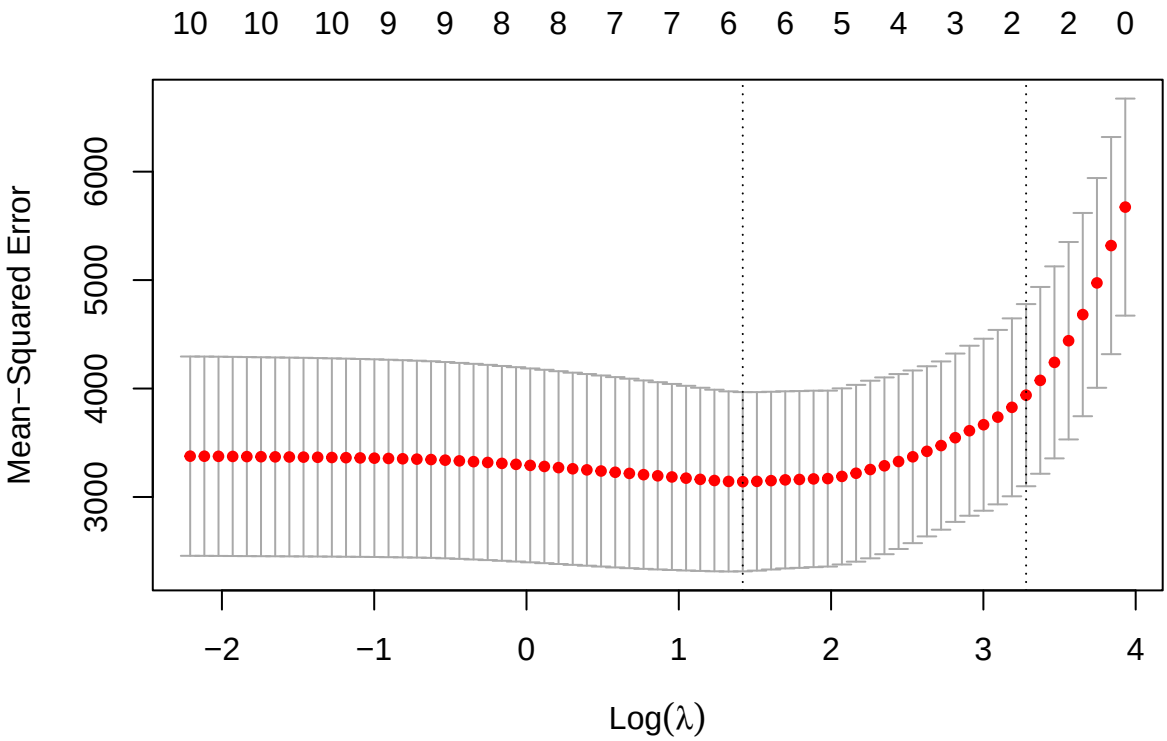
```
##
## Call:
## lm(formula = y ~ x2 + x4 + x5 + x8 + x9 + x10, data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -225.56  -19.99    0.99   26.95  160.64
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 116.0009    35.9586   3.226  0.00147 **
## x2            0.5461     0.2522   2.165  0.03161 *
## x4           -2.2289     0.5215  -4.274 3.01e-05 ***
## x5            1.1608     0.4931   2.354  0.01957 *
## x8           -0.8277     0.3587  -2.308  0.02207 *
## x9            1.2466     0.4980   2.503  0.01314 *
## x10           1.0212     0.2316   4.409 1.73e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 49.95 on 193 degrees of freedom
## Multiple R-squared:  0.5772, Adjusted R-squared:  0.5641
## F-statistic: 43.92 on 6 and 193 DF,  p-value: < 2.2e-16
```

```r
summary(lm.alt.stepback)
```

```
##
## Call:
## lm(formula = y ~ x1 + x3 + x5 + x8 + x9, data = d2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -109.598  -19.563   -0.195   21.607  121.755
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  95.0097    44.1424   2.152  0.03266 *
## x1           -0.4076     0.2715  -1.501  0.13504
## x3            3.4611     0.4381   7.901 2.35e-13 ***
## x5            2.0362     0.1284  15.852  < 2e-16 ***
## x8           -1.5114     0.5325  -2.838  0.00504 **
## x9            0.9598     0.3810   2.519  0.01260 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 37.29 on 186 degrees of freedom
## Multiple R-squared:  0.6547, Adjusted R-squared:  0.6455
## F-statistic: 70.55 on 5 and 186 DF,  p-value: < 2.2e-16
```

```r
lambda_grid <- 10^ seq(10 , -3 , length =200)

lasso.full_model <- glmnet(as.matrix(d[,-ncol(d)]), d$y, alpha=1, lambda=lambda_grid, standardize = TRU
```

```r
lasso.full.cv.out <- cv.glmnet(as.matrix(d[,-ncol(d)]), d$y, alpha=1)
plot(lasso.full.cv.out)
```



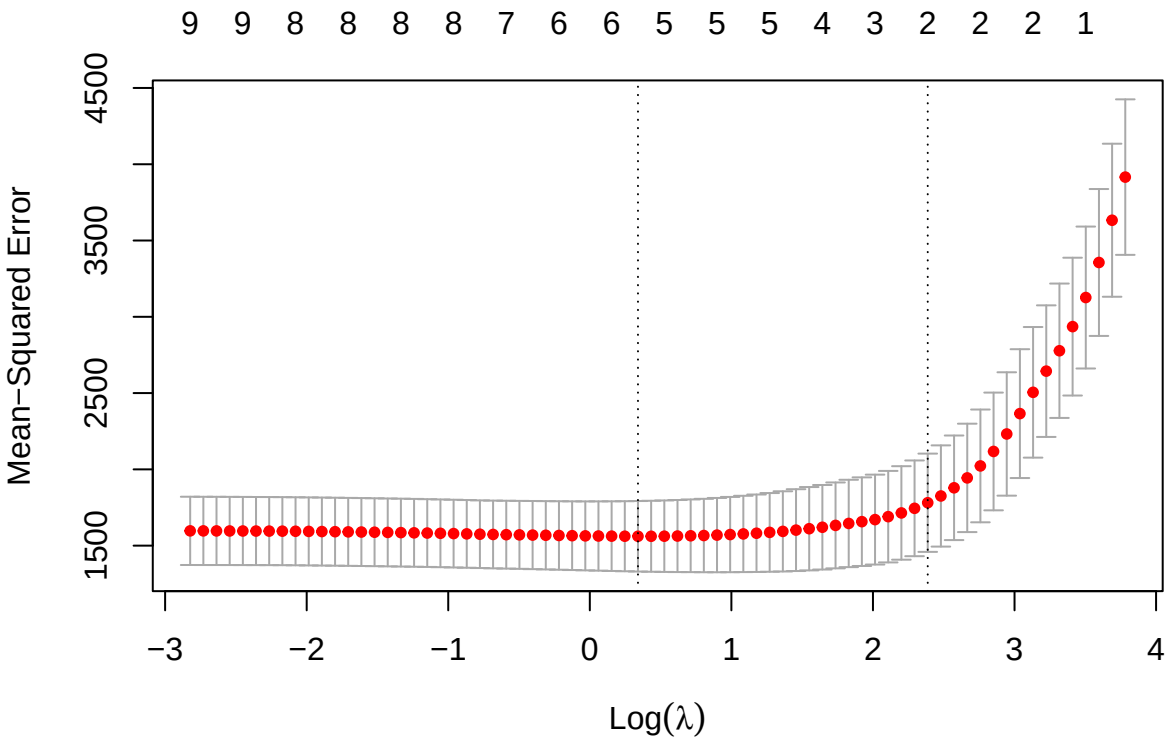

```r
lasso.full.lambda <- lasso.full.cv.out$lambda.min
lasso.full.coef <- predict(lasso.full_model, type = "coefficients", s = lasso.full.lambda)
print("Lasso full")
```

```
## [1] "Lasso full"
```

```r
print(as.matrix(lasso.full.coef)[seq(2,length(d)),] != 0)
```

```
##    x1    x2    x3    x4    x5    x6    x7    x8    x9   x10
## FALSE  TRUE FALSE  TRUE  TRUE FALSE FALSE  TRUE  TRUE  TRUE
```

```r
lasso.alt_model <- glmnet(as.matrix(d2[,-ncol(d2)]), d2$y, alpha=1, lambda=lambda_grid, standardize = TR

lasso.alt.cv.out <- cv.glmnet(as.matrix(d2[,-ncol(d2)]), d2$y, alpha=1)
plot(lasso.alt.cv.out)
```




```
lasso.alt.lambda <- lasso.alt.cv.out$lambda.min
lasso.alt.coef <- predict(lasso.alt_model, type = "coefficients", s = lasso.alt.lambda)
print("Lasso alt")
```

```
## [1] "Lasso alt"
```

```
print(as.matrix(lasso.alt.coef)[seq(2,length(d2)),] != 0)
```

```
##    x1    x3    x4    x5    x6    x7    x8    x9   x10
##  TRUE  TRUE FALSE  TRUE FALSE FALSE  TRUE  TRUE FALSE
```

Add your verbal summary including the final wrap up here (1p): From step-wise selections I chose backwards selection for both, since it doesn't add much RSS, but simplifies the model.
Then I compare these models with models from LASSO. For the both models, LASSO chooses same parameters as chosen step-wise selection.

Final model I choose is described with equation
y ~ -0.4076 * x1 + 3.4611 * x3 + 2.0362 * x5 - 1.5114 * x8 + 0.9598 * x9

Grading

- 1p for selection of proper methods for assumption verification and their explanation
- 1p for the correct implementation of issue removal
- 1p for its explanation and right decisions in further model design themselves
- 1p for proper application of feature selection techniques and their comparison
- 1p for comprehensive evaluation of all the model improvements and the final wrap up